

Learning Hierarchical Navigable Small Worlds (HNSW) Graph Construction via Query Supervision

Miloni Dipak Thakkar
miloni.thakkar@iitgn.ac.in
IIT Gandhinagar
Gandhinagar, Gujarat, India

Khushi Garg
khushi.garg@iitgn.ac.in
IIT Gandhinagar
Gandhinagar, Gujarat, India

Beli Nilesh Dedhia
beli.dedhia@iitgn.ac.in
IIT Gandhinagar
Gandhinagar, Gujarat, India

Ankit Saha
ankit.saha@iitgn.ac.in
IIT Gandhinagar
Gandhinagar, Gujarat, India

Abstract

Hierarchical Navigable Small Worlds (HNSW) is the state-of-the-art algorithm for approximate nearest neighbor (ANN) search, underpinning modern vector databases and retrieval systems. Its graph is constructed using fixed heuristics governed by parameters—the number of connections per node M , the construction beam width ef_{con} , and the layer assignment coefficient m_L —whose values are chosen manually without reference to the actual query workload or data distribution. This project asks: given a set of ANN query examples and their outputs, can we learn the parameter values that govern HNSW graph construction so that the resulting graph is optimized for recall and latency? We propose three complementary approaches: learning the global construction parameters via Bayesian optimization, learning the layer assignment function from query trace supervision, and learning the neighbor selection criterion as a parameterized scoring function. We evaluate on standard ANN benchmarks using recall@10 and queries-per-second as primary metrics.

CCS Concepts

• **Information systems** → Nearest-neighbor search; • **Computing methodologies** → Bayesian network models; • **Mathematics of computing** → Graph algorithms.

Keywords

Approximate Nearest Neighbor Search, HNSW, Graph Construction, Bayesian Optimization, Learned Index Structures, Parameter Learning, Query Traces

ACM Reference Format:

Miloni Dipak Thakkar, Beli Nilesh Dedhia, Khushi Garg, and Ankit Saha. 2026. Learning Hierarchical Navigable Small Worlds (HNSW) Graph Construction via Query Supervision. In *Proceedings of CS328: Introduction to Data Science (CS328 Project Proposal)*. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Approximate Nearest Neighbor (ANN) search is a core primitive in modern machine learning—powering semantic search, recommendation systems, and retrieval-augmented generation. Among ANN algorithms, Hierarchical Navigable Small Worlds (HNSW) [5]

achieves the best known recall-versus-latency trade-off and underlies production systems such as Pinecone, Weaviate, and Qdrant.

HNSW organizes data into a multi-layered proximity graph. A sparse top layer enables fast long-range navigation; denser lower layers refine search locally. Graph construction is controlled by three parameters— M (edges per node), ef_{con} (construction beam width), and m_L (layer assignment coefficient)—alongside two internal heuristics: a fixed exponential distribution for layer assignment, and a greedy geometric criterion for neighbor selection. None of these adapt to the data distribution or query workload; they are set by hand using documentation defaults.

This project addresses the question from the project brief directly: *given a set of ANN query examples and their outputs, can we learn the graph structure used in HNSW by parameterizing graph creation and learning the parameter values based on final metrics?* We treat the global parameters and the internal structural decisions of HNSW as learnable quantities, using query examples with their ground-truth outputs as supervision, and measuring success through final recall and latency metrics.

2 Related Work

HNSW [5] builds upon Navigable Small World graphs [4] by adding a hierarchical layer structure for efficient navigation. Its construction heuristics have remained fixed since the original paper. Peng et al. [6] use learned ranking to prune candidates *during search*, reducing distance computations—but the underlying graph structure is never modified. Kraska et al. [3] demonstrate that ML models can replace traditional data structure heuristics, motivating our learning-based approach to graph construction. Snoek et al. [8] establish Bayesian optimization as the standard method for hyperparameter learning when objective evaluation is expensive, which we apply directly to HNSW’s parameter space. Our work is the first to target all three levels of HNSW’s construction—global parameters, layer assignment, and neighbor selection—and learn them jointly from query workload supervision.

3 Methodology

We propose three complementary approaches to learning HNSW’s graph construction. Each targets a different level of the construction process, and together they form a unified learned pipeline.

3.1 Learning Global Construction Parameters

The parameters M , ef_{con} , and m_L directly define the graph’s overall structure—how many edges each node has, how carefully neighbors are chosen during construction, and how nodes are distributed across layers. Currently these are set by hand using documentation defaults. We treat the mapping from parameter values to final recall and QPS as a black-box function and apply Bayesian optimization (via *optuna*) to find the configuration that maximizes recall subject to a latency budget, using training query examples and their ground-truth outputs as the evaluation signal.

We compare the learned configuration against vanilla HNSW with defaults, grid search over the same space, and random search—to isolate the specific benefit of Bayesian optimization as the learning mechanism.

3.2 Learning the Layer Assignment Function

In standard HNSW, each node’s layer is assigned by drawing from a fixed exponential distribution parameterized by m_L —a purely random process with no reference to a node’s structural role in the data. We replace this fixed distribution with a learned scoring function. We collect query traces from a preliminary index, recording which nodes are visited during successful searches across training queries. Nodes that appear frequently in successful search paths are natural routing nodes and benefit from placement in higher layers. We use these traversal signals as supervision to train a model that predicts, from node features, how valuable a node is as a higher-layer routing point. The predicted scores replace the random draw in layer assignment, making the hierarchy data-driven rather than random.

3.3 Learning the Neighbor Selection Criterion

HNSW currently selects each node’s neighbors using a fixed geometric heuristic—essentially connecting to the closest available nodes with some diversity enforcement. We parameterize this decision as a learned scoring function over node and edge features, including pairwise distance, local density, traversal frequency from query traces, and directional diversity relative to existing connections. The weights of this scoring function are learned from training query examples to maximize recall, replacing the fixed heuristic with a data-driven criterion for which edges to form.

3.4 Combined Pipeline

The three approaches compose into a unified pipeline: global parameters define the graph’s overall shape, the learned layer assignment determines the hierarchy, and the learned neighbor selection refines connectivity at each layer. A key research question is whether the three levels reinforce each other—and whether learning all three jointly from query data yields better results than optimizing each in isolation.

4 Datasets and Libraries

We evaluate on three standard benchmarks from the *ann-benchmarks* suite [1], covering different dimensionalities, data types, and geometric properties. Each dataset provides pre-computed ground-truth nearest neighbors for evaluation. We use 80% of queries for training and 20% as a held-out test set.

Table 1: Evaluation Datasets

Dataset	Dim	Base Size	Queries	Metric
SIFT1M [2]	128	1,000,000	10,000	L_2
GloVe-100 [7]	100	1,183,514	10,000	Cosine
Fashion-MNIST	784	60,000	10,000	L_2

Our primary evaluation metric is the **recall-QPS frontier**: for each method, we vary ef_{search} and plot the resulting curve in (QPS, Recall@10) space. A method is better if its curve is upper-right—higher recall at the same speed or same recall at higher speed. This is the standard evaluation in ANN research [1]. We additionally report index build time, memory usage, and edge count.

Key libraries include *hnswlib* for HNSW index construction and search, *optuna* for Bayesian optimization, *scikit-learn* and *xgboost* for learned scoring functions, and *matplotlib* for plotting. All experiments run on CPU to ensure fair latency comparisons. Code is maintained on GitHub.

5 Timeline

- **Week 1 (Mar 25 – Apr 1):** Environment setup, dataset loading, evaluation harness, vanilla HNSW baseline.
- **Week 2 (Apr 1 – Apr 8):** Bayesian optimization of global parameters. *Biweekly Update 1*.
- **Week 3 (Apr 8 – Apr 16):** Query trace collection, learned layer assignment, learned neighbor selection.
- **Week 4 (Apr 16 – Apr 23):** Full pipeline evaluation, ablations, report writing. *Biweekly Update 2*.

Acknowledgments

The authors thank Prof. Anirban Dasgupta for the project guidance and problem formulation.

References

- [1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems* 87 (2020), 101374.
- [2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [3] Tim Kraska, Alex Beutel, Ed H Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The case for learned index structures. In *Proceedings of the 2018 ACM SIGMOD International Conference on Management of Data*. 489–504.
- [4] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [5] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2018), 824–836.
- [6] Yun Peng, Byron Choi, Tsz Nam Chan, and Jianliang Xu. 2022. LAN: Learning-based approximate k -nearest neighbor search in graph databases. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. IEEE, 2508–2521.
- [7] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1532–1543.
- [8] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. 2012. Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, Vol. 25.